LAPORAN PRAKTIKUM PEKAN 3

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU :

Dr. WAHYUDI, S.T., M.T.

OLEH :

MUHAMMAD HANS NAFIS

NIM : 2511532027

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kepada Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 3 ini tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu tugas praktikum mata kuliah Basis Data, dengan topik pembahasan mengenai konsep dasar *Stack*.

Dalam laporan ini, penulis menyajikan pembahasan yang sistematis dan mudah dipahami, mulai dari konsep dasar *Stack*, prinsip kerja, hingga implementasinya dalam pemrograman, Praktikum ini memberikan pengalaman langsung kepada penulis dalam menerapkan teori yang telah dipelajari di perkuliahan ke dalam bentuk praktik, sehingga pemahaman menjadi lebih konkret.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, asisten dosen, teman-teman praktikum dan pihak lain yang turut mendukung penulisan laporan ini.

Kamis, 09 April 2026

Muhammad Hans Nafis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB 1 1

PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB 2 2

PEMBAHASAN 2

2.1 *Stack* 2

2.2 Class StackArray 2

2.3 Class StackArrayDriver 5

2.4 Class StackPostfix 6

2.5 Class NilaiMaksimum 8

2.6 Class SiswaStack 10

BAB 3 14

KESIMPULAN 14

DAFTAR PUSTAKA 15

LAMPIRAN 16

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu komponen penting dalam ilmu komputer yang berperan dalam pengorganisasian data agar dapat diakses dan dikelola secara efisien. Salah satu jenis struktur data yang sederhana namun sangat penting adalah *Stack*. *Stack* merupakan struktur data linear yang menggunakan prinsip *Last In First Out* (LIFO), di mana elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan.

Konsep *Stack* banyak digunakan dalam berbagai aplikasi komputer, seperti pengelolaan memori, evaluasi ekspresi matematika, proses pemanggilan fungsi, serta fitur *undo* dan *redo* dalam aplikasi. Oleh karena itu, pemahaman terhadap *Stack* menjadi sangat penting bagi mahasiswa yang mempelajari bidang informatika atau ilmu komputer.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar *Stack*.
2. Mempelajari operasi-operasi dasar pada *Stack*, seperti *pop*, *peek*, dan *isEmpty*.
3. Mengimplementasikan struktur data *Stack* dalam bahasa pemrograman.

1.3 Manfaat Praktikum

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut :

1. Menambah pemahaman mengenai konsep dasar *Stack*.
2. Melatih keterampilan dalam mengimplementasikan konsep *Stack* ke dalam bahasa pemrograman Java.
3. Memahami penggunaan metode – metode penting pada *Stack*.

BAB 2
PEMBAHASAN

2.1 *Stack*

Stack merupakan salah satu struktur data linear yang memiliki karakteristik khusus, yaitu menggunakan pronsip *Last In First Out* (LIFO). Prinsip ini berarti elemen terakhir yang dimasukkan ke dalam *Stack* akan menjadi elemen pertama yang dikeluarkan. Dalam implementasinya, seluruh proses penambahan dan penghapusan elemen hanya dilakukan pada satu sisi saja, yaitu bagian atas (*top*) dari *Stack*. Konsep ini dapat dianalogikan seperti tumpukan piring, di mana piring yang terakhir diletakkan di atas akan menjadi yang pertama diambil.

Operasi dasar dalam steuktur data *Stack* terdiri dari beberapa fungsi utama, yaitu *push*, *pop*, dan *peek*. Operasi *push* digunakan untuk menambahkan elemen ke dalam *Stack* pada posisi paling atas, sedangkan *pop* digunakan untuk menghapus elemen teratas dari *Stack*. Selain itu, terdapat operasi *peek* atau *top* yang berfungsi untuk melihat eelemen paling atas tanpa menghapusnya, serta *isEmpty* untuk mengecek apakah *Stack* dalam keadaan kosong. Operasi-operasi ini menjadi dasar dalam pengelolaan data menggunakan *Stack* dan umumnya memiliki kompleksitas waktu yang efisien.

Dalam implementasinya, *Stack* dapat dipresentasikan menggunakan beberapa cara, seperti *array* maupun *linked list*. Pada implementasi berbasis *array*, *Stack* memiliki ukuran tertentu sehingga berpotensi mengalami kondisi *overflow* ketika kapasitas penuh. Sedangkan pada *linked list*, ukuran *Stack* bersifat dinamis sehingga lebih fleksibel. Selain itu, terdapat juga kondisi *underflow*, yaitu ketika operasi *pop* dilakukan pada *Stack* yang kosong. Pemilihan metode implementasi ini bergantung pada kebutuhan dan efisiensi yang diinginkan dalam suatu program.

2.2 Class StackArray

Berikut kode program Java mengenai class StackArray :

```
public class StackArray_2511532027 {
	static final int MAX_2027 = 1000;
	int top_2027;
	int a_2027[] = new int [MAX_2027];
	boolean isEmpty_2027()
	{
```

```
        return (top_2027 < 0);
    }
    StackArray_2511532027()
    {
        top_2027 = -1;
    }
    boolean push_2027 (int x)
    {
        if (top_2027 >= (MAX_2027 - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a_2027[++top_2027] = x;
            System.out.println(x + " ditambahkan dalam stack");
            return true;
        }
    }
    int pop_2027()
    {
        if (top_2027 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a_2027[top_2027--];
            return x;
        }
    }
    int peek_2027()
    {
        if (top_2027 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a_2027[top_2027];
            return x;
```

```
            }
        }
        void print_2027() {
            for (int i = top_2027; i > -1; i--) {
                System.out.print(" " + a_2027[i]);
            }
        }
}
```

Kode program tersebut merupakan implementasi struktur data *Stack* menggunakan *array* dalam bahasa pemrograman Java. Kelas StackArray_2511532027 memiliki konstanta MAX_2027 yang berfungsi untuk menentukan kapasitas maksimum *Stack*, yaitu sebanyak 1000 elemen. Selain itu, terdapat variabel top_2027 yang digunakan sebagai penunjuk posisi elemen teratas dalam *Stack*, serta *array* a_2027[] yang berfungsi sebagai tempat penyimpanan data. Pada konstruktor, nilai top_2027 diinisialisasi dengan -1 yang menandakan bahwa *Stack* dalam keadaan kosong.

Method isEmpty_2027() digunakan untuk memeriksa apakah *Stack* kosong atau tidak. Method ini akan mengembalikan nilai *true* jika top_2027 bernilai kurang dari 0, yang berarti belum ada elemen yang dimasukkan ke dalam *Stack*. Sebaliknya, jika nilai top_2027 lebih dari atau sama dengan 0, maka *Stack* dianggap tidak kosong. Method ini sangat penting untuk mencegah kesalahan saat melakukan operasi pengambilan data.

Method push_2027(int x) digunakan untuk menambahkan elemen ke dalam *Stack*. Sebelum menambahkan elemen, method ini terlebih dahulu memeriksa apakah *Stack* sudah penuh dengan membandingkan nilai top_2027 dengan kapasitas maksimum (MAX_2027 – 1). Jika *Stack* penuh, maka akan ditampilkan pesan “Stack Overflow” dan operasi tidak dilakukan. Namun, jika masih tersedia ruang, maka nilai top_2027 akan dinaikkan terlebih dahulu, kemudian elemen x dimasukkan ke dalam *array* pada posisi tersebut. Hal ini menunjukkan penerapan prinsip LIFO, di mana elemen baru selalu ditambahkan ke bagian atas *Stack*.

Method pop_2027() berfungsi untuk mengapu dan mengembalikan elemen teratas dari *Stack*. Method ini akan terlebih dahulu memeriksa apakah *Stack* kosong. Jika kosong, maka akan ditampilkan pesan “Stack Underflow” dan mengembalikan nilai 0 sebagai tanda tidak ada data yang bisa diambil. Jika tidak kosong, maka

elemen pada posisi top_2027 akan disimpan ke dalam variabel sementara, kemudia nilai top_2027 akan dikurangi satu untuk menghapus elemen tersebut dari *Stack*.

Method peek_2027() digunakan untuk melihat elemen teratas tanpa menghapusnya dari *Stack*. Cara kerjanya hampir sama denga method pop_2027(), yaitu dengan melakukan pengecekan apakah *Stack* kosong atau tidak. Jika tidak kosong, maka elemen pada posisi top_2027 akan dikembalikan tanpa mengubah nilai top_2027, sehingga data tetap berada di dalam *Stack*.

Method print_2027() digunakan untuk menampilkan seluruh isi *Stack*. Method ini menggunakan perulangan dari indeks top_2027 hingga ke indeks 0, sehingga elemen ditampilkan dari atas ke bawah. Hal ini sesuai dengan konsep *Staxk* yang menampilkan elemen terakhir terlebih dahulu.

2.3 Class StackArrayDriver

Berikut kode program Java mengenai class StackArrayDriver :

```
public class StackArrayDriver_2511532027 {
	public static void main(String[] args) {
		StackArray_2511532027 s = new StackArray_2511532027();
		s.push_2027(10);
		s.push_2027(20);
		s.push_2027(30);
		System.out.println(s.pop_2027() + " dikeluarkan dari
stack");
		System.out.println("Elemen teratas adalah : " +
s.peek_2027());
		System.out.print("Elemen pada stack : ");
		s.print_2027();
	}
}
```

Program di atas merupakan kelas driver yang digunakan untuk menguji implementasi struktur data *Stack* yang telah dibuat pada kelas StackArray_2511532027. Pada method main, dibuat objek s dari kelas StaxkArray_2511532027 yang akan digunakan untuk menjalankan berbagai operasi *Stack* seperti penambahan, penghapusan, dan penampilan elemen.

Selanjutnya, program melakukan beberapa operasi *push* yaitu s.push_2027(10), s.push_2027(20), dan s.push_2027(30). Operasi ini bertujuan

untuk menambahkan elemen ke dalam *Stack* secara berurutan. Karena *Stack* menggunakan prinsip *Last In First Out* (LIFO), maka elemen terakhir yang dimasukkan, yaitu 30, akan berada di posisi paling atas (*top*) dari *Stack*.

Setelah itu, program menjalankan operasi pop_2027() yang berfunhsi untuk menghapus elemen teratas dari *Stack*. Hasil dari operasi ini kemudian ditampilkan dengan pesan bahwa elemen tersebut telah dikeluarkan dari *Stack*. Dalam kasus ini, elemen yang dihapus adalah 30 karena merupakan elemen terakhir yang dimasukkan.

Kemudian, method peek_2027() dipanggil untuk melihat elemen yang berada di posisi teratas setelah proses *pop* dilakukan. Karena elemen 30 telah dihapus, maka elemen teratas yang tersisa adalah 20. Nilai ini ditampilkan ke layar tanpa menghapusnya dari *Stack*. Terakhir, method print_2027() digunakan untuk menampilkan seluruh isi *Stack* yang tersisa. Elemen akan ditampilkan dari atas ke bawah. Dalam kondisi akhir, *Stack* berisi elemenn20 dan 10, sehingga output yang ditampilkan mencerminkan urutan tersebut.

2.4 Class StackPostfix

Berikut kode program Java mengenai class StackPostfix :

```
import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511532027 {
	public static int postfixEvaluate(String expression_2027) {
		Stack<Integer> s_2027 = new Stack<Integer>();
		Scanner input_2027 = new Scanner(expression_2027);
		while (input_2027.hasNext()) {
			if (input_2027.hasNextInt()) { // an operand
(integer)
				s_2027.push(input_2027.nextInt());
			} else { // an operator
				String operator_2027 = input_2027.next();
				int operand2_2027 = s_2027.pop();
				int operand1_2027 = s_2027.pop();
				if (operator_2027.equals("+")) {
					s_2027.push(operand1_2027 +
operand2_2027);
```

```
                        } else if (operator_2027.equals("-")) {
                                s_2027.push(operand1_2027 -
operand2_2027);
                        } else if (operator_2027.equals("*")) {
                                s_2027.push(operand1_2027 *
operand2_2027);
                        } else {
                                s_2027.push(operand1_2027 /
operand2_2027);
                        }
                }
            }
            input_2027.close();
            return s_2027.pop();
        }
        public static void main(String[] args) {
            System.out.println("Hasil postfix = " + postfixEvaluate("5
2 4 * + 7 -"));
        }
}
```

Program di atas merupakan implementasi penggunaan struktur data *Stack* untuk melakukan evaluasi ekspresi dalam bentuk *postfix*. Pada bagian awal, progrsm mengimpor library Scanner dan Stack dari Java, di mana Stack digunakan sebagai struktur data utama untuk menyimpan operand sementara selama proses perhitungan berlangsung.

Method postfixEvaluate(String expression_2027) merupakan fungsi utama yang digunakan untuk menghitung hasil dari ekspresi postfix. Di dalam method ini, dibuat sebuah objek *Stack* bernama s_2027 dengan tipe data integer. Selain itu, digunakan objek Scanner untuk membaca ekspresi yang diberikan dalam bentuk string. Scanner ini akan memisahkan setiap elemen dalam ekspresi berdasarkan spasi, sehingga dapat diproses satu per satu.

Proses evaluasi dilakukan menggunakan perulangan while yang berjakan selama masih ada token dalam ekspresi. Jika token yang dibaca merupakan bilangan (*operand*), maka nilai tersebut langsung dimasukkan ke dalam *Stack* menggunakan operasi *push*. Namun, jika token yang dibaca merupakan operator

seperti +, -, *, atau /, maka program akan mengambil dua elemen teratas dari stack menggunakan iterasi *pop*. Elemen pertama yang diambil menjadi operand kedua, dan elemen berikutnya menjadi operand pertaa. Hal ini penting karena urutan operasi dalam postfix harus diperhatikan.

Setelah dua operand diambil, program akan melakukan operasi sesuai dengan operator yang dibaca. Jika operator adalah penjumlahan, maka kedua operand ditambahkan; jika pengurangan, maka kedua operand dikurangkan; jika perkalian, maka kedua operand dikalikan; dan jika pembagian, maka kedua operand pertama dibagi dengan operand kedua. Hasil dari operasi tersebut kemudian dimasukkan kembali ke dalam *Stack*. Proses ini terus berulang hingga seluruh ekspresi selesai diproses.

Setelah perulangan selesai, nilai akhir yang tersisa di dalam *Stack* merupakan hasil dari evaluasi ekspresi postfix. Nilai ini kemudian diambil menggunakan operasi *pop* dan dikembalikan sebagai hasil dari method. Scanner yang digunakan juga ditutup untuk menghindari kebocoran sumber daya.

Pada method main, program memanggil fungsi postfixEvaluate dengan contoh ekspresi "5 2 4 * + 7 –". Ekspresi tersebut akan dihitung menggunakan aturan postfix, yaitu dengan mengalikan 2 dan 4 terlebih dahulu, kemudian menambahkan hasilnya dengan 5, dan terakhir menguranginya dengan 7. Hasil akhir dari perhitungan tersebut kemudian ditampilkan ke layar.

2.5 Class NilaiMaksimum

Berikut kode program Java mengenai class NilaiMaksimum :

```
import java.util.Stack;

public class NilaiMaksimum_2511532027 {
	public static int max(Stack<Integer> s_2027) {
		Stack<Integer> backup_2027 = new Stack<Integer>();
		int maxValue_2027 = s_2027.pop();
		backup_2027.push(maxValue_2027);
		while (!s_2027.isEmpty()) {
			int next = s_2027.pop();
			backup_2027.push(next);
			maxValue_2027 = Math.max(maxValue_2027, next);
		}
```

```
            while (!backup_2027.isEmpty()) {
                s_2027.push(backup_2027.pop());
            }
            return maxValue_2027;
        }
        public static void main (String[] args) {
            Stack<Integer> s_2027 = new Stack<Integer>();
            s_2027.push(70);
            s_2027.push(12);
            s_2027.push(20);
            System.out.println("Isi Stack " + s_2027);
            System.out.println("Stack Teratas " + s_2027.peek());
            System.out.println("Nilai Maksimum " + max(s_2027));
        }
}
```

Program di atas merupakan implementasi penggunaan struktur data *Stack* untuk mencari nilai maksimum dari elemen-elemen yang terdapat di dalam *Stack* tanpa menghilangkan isi aslinya. Program ini memanfaatkan kelas Stack dari Java sebagai struktur utama dalam penyimpanan data serta menggunakan *Stack* tambahan untuk menjaga agardata tidak hilang selama proses pencarian nilai maksimum.

Pada method max(Stack<Integer> s_2027), dibuat sebuah *Stack* baru bernama backup_2027 yang berfungsi sebagai tempat penyimpanan sementara. Kemudian, elemen teratas dari *Stack* utama diambil menggunakan operasi *pop* dan disimpan sebagai nilai awal maxValue_2027. Nilai tersebut juga dimasukkan ke dalam *Stack* cadangan. Selanjutnya, program melakukan perulangan selama *Stack* utama belum kosong. Pada setiap iterasi, elemen berikutnya diambil dari *Stack* utama, dimasukkan ke dalam *Stack* cadangan, dan dibandingkan dengan nilai maksimum sementara menggunakan fungsi Math.max(). Jika nilai tersebut lebih besar, maka nilai maksimum akan diperbarui.

Setelah seluruh elemen dalam *Stack* utama diproses dan nilai maksimum ditentukan, langkah berikutnya adalah mengembalikan isi *Stack* ke kondisi semula. Hal ini dilakukan dengan memindahkan kembali seluruh elemen dari *Stack* cadangan ke *Stack* utama menggunakan operasi *pop* dan *push*. Dengan cara ini, urutan dan isi *Stack* tetap sama seperti sebelum proses pencarian dilakukan.

Pada method main, program membuat sebuah *Stack* dan mengisi beberapa nilai, yaitu 70,12, dan 20. Kemudian, isi *Stack* ditampilkan, diikuti dengan elemen teratas menggunakan metod peek(). Setelah itu, method max() dipanggil untuk mencari nilai maksimum dari *Stack* tersebut, dan hasilnya ditampilkan ke layar.

2.6 Class SiswaStack

Berikut kode program Java mengenai class SiswaStack :

```
import java.util.ArrayList;
class Siswa_2027 {
	String nama_2027;
	int nim_2027;

	public Siswa_2027(String nama_2027, int nim_2027) {
		this.nama_2027 = nama_2027;
		this.nim_2027 = nim_2027;
	}

	@Override
	public String toString() {
		return "Nim : " + nim_2027 + ", Nama : " + nama_2027;
	}
}
public class SiswaStack_2511532027 {
	private ArrayList<Siswa_2027> stack_2027;

	public SiswaStack_2511532027() {
		stack_2027 = new ArrayList<>();
	}

	public void push_2027(Siswa_2027 mhs_2027) {
		stack_2027.add(mhs_2027);
	}

	public Siswa_2027 pop_2027() {
		if (!isEmpty()) {
			return stack_2027.remove(stack_2027.size() - 1);
		}
		return null;
```

```java
        }

        public Siswa_2027 peek_2027() {
                if (!isEmpty()) {
                        return stack_2027.get(stack_2027.size() - 1);
                }
                return null;
        }

        public boolean isEmpty() {
                return stack_2027.isEmpty();
        }

        public void tampilkanMahasiswa_2027() {
                for (int i = stack_2027.size() - 1; i >= 0; i--) {
                        System.out.println(stack_2027.get(i));
                }
        }
        public static void main(String[] args) {
                SiswaStack_2511532027 studentStack_2027 = new
SiswaStack_2511532027();

                Siswa_2027 mhs1_2027 = new Siswa_2027("Ali", 1);
                Siswa_2027 mhs2_2027 = new Siswa_2027("Boby", 2);
                Siswa_2027 mhs3_2027 = new Siswa_2027("Charles", 3);

                studentStack_2027.push_2027(mhs1_2027);
                studentStack_2027.push_2027(mhs2_2027);
                studentStack_2027.push_2027(mhs3_2027);

                System.out.println("Siswa di dalam stack : ");
                studentStack_2027.tampilkanMahasiswa_2027();

                System.out.println("Siswa teratas " +
studentStack_2027.peek_2027());
                System.out.println("Mengeluarkan siswa teratas dari stack :
" + studentStack_2027.pop_2027());
                System.out.println("Daftar siswa setelah di pop : ");
```

```
            studentStack_2027.tampilkanMahasiswa_2027();
        }
}
```

Program di atas merupakan implementasi struktur data *Stack* menggunakan ArrayList dengan objek bertipe data khusus, yaitu Siswa_2027. Program ini terdiri dari dua kelas utama, yaitu kelas Siswa_2027 yang mempresentasikan data mahasiswa, dan kelas SiswaStack_2511532027 yang berfungsi sebagai struktur *Stack* untuk menyimpan objek-objek tersebut.

Kelas Siswa_2027 memiliki dua atribut, yaitu nama_2027 dan nim_2027, yang masing-masing digunakan untuk menyimpan nama dan nomor induk mahasiswa. Kelas ini juga memiliki konstruktor untuk menginisialisasi nilai kedua atribut tersebut saat objek dibuat. Selain itu, method toString() di *override* untuk mengembalikan representasi string dari objek, sehingga ketika objek dicetak, akan menampilkan informasi NIM dan nama mahasiswa secara rapi.

Pada kelas SiswaStack_2511532027, digunakan ArrayList<Siswa_2027> sebagai media penyimpanan *Stack*. Meskipun ArrayList pada dasarnya adalah struktur data dinamis, dalam program ini ia dimanfaatkan untuk meniru perilaku *Stack* dengan mengakses elemen terakhir sebagai bagian atas (*top*). Konstruktor kelas ini menginisialisasi stack_2027 sebagai ArrayList kosong.

Method push_2027() digunakan untuk menambahkan objek Siswa_2027 ke dalam *Stack*. Proses ini dilakukan dengan menambahkan elemen ke akhir ArrayList. Method pop_2027() digunakan untuk menghapus dan mengembalikan elemen teratas dari *Stack*, yaitu elemen pada indeks terakhir. Sebelum melakukan operasi ini, dilakukan pengecekan apakah *Stack* kosong menggunakan method isEmpty() untuk menghindari kesalahan. Jika *Stack* kosong, method akan mengembalikan nilai null.

Selanjutnya, method peek_2027() digunakan untuk emlihat elemen teratas tanpa menghapusnya. Sama seperti pop_2027(), method ini juga melakukan pengecekan kondisi kosong terlebih dahulu. Method isEmpty() sendiri berfungsi untuk memeriksa apakah *Stack* dalam keadaan kosong dengan memanfaatkan method bawaan isEmpty() dari ArrayList.

Method tampilkanMahasiswa_2027() digunakan untuk menampilkan seluruh isi *Stack*. Perulangan dilakukan dari indeks terakhir hingga indeks pertama,

sehingga elemen ditampilkan dari atas ke bawah sesuai dengan konsep *Stack*. Hal ini memperlihatkan bahwa elemen yang terakhir dimasukkan akan ditampilkan terlebih dahulu.

Pada method main, program membuat beberapa objek Siswa_2027 dengan data yang berbeda, kemudian menambahkannya ke dalam *Stack* menggunakan method push_2027(). Setelah itu, isi *Stack* ditampilkan, diikuti dengan penampilan elemen teratas menggunakan peek_2027(). Program juga melakukan operasi pop_2027() untuk menghapus elemen teratas, kemudian menampilkan kembali isi *Stack* setelah penghapusan.

BAB 3
KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Stack* merupakan salah satu struktur data linear yang memiliki peran penting dalam pengolahan data dengan prinsip *Last In First Out* (LIFO). Melalui berbagai implementasi yang telah dibuat, baik menggunakan *array*, class bawaan Stack, maupun ArrayList, dapat dipahami bahwa *Stack* memiliki operasi dasar seperti *push*, *pop*, *peek*, dan *isEmpty* yang sanga fundamental dalam pengelolaan data. Dengan demikian,pemahaman terhadap konsep dan implementasi *Stack* sangat penting sebagai dasar dalam mempelajari struktur data dan algoritma yang lebih lanjut.

Sebagai saran, diharapkan mahasiswa untuk lebih banyak melakukan latihan dengan variasi kasus yang berbeda, seperti implementasi *Stack* menggunakan struktur data lain atau penerapan pada permasalahan nyata yang lebih kompleks. Selain itu, penting untuk memahami konsep dasar secara mendalam sebelum mengimplementasikannya dalam kode program agar dapat meminimalisir kesalahan logika. Dengan adanya latihan yang konsisten dan pemahaman yang baik, diharapkan mahasiswa dapat menguasai konsep *Stack* secara optimal dan mampu mengaplikasikannya dalam pengembangan program yang lebih kompleks.

DAFTAR PUSTAKA

[1] https://www.geeksforgeeks.org/java/stack-class-in-java/ [Diakses 09 April 2026]

[2] https://www.programiz.com/java-programming/stack [Diakses 09 April 2026]

[3] https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html [Diakses 09 April 2026]

LAMPIRAN